

**Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ**

**Лабораторная работа №7
По дисциплине «Современные платформы программирования»
Тема: «Phyton тестирование»**

Выполнил:
Студент 3 курса
Группы ПО-12
Боричевский Д. О.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Требования к оформлению отчета

Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями):

- 1) Изложение цели работы.
- 2) Задание по лабораторной работе с описанием своего варианта.
- 3) Спецификации ввода-вывода программы.
- 4) Текст программы (кратко).
- 5) Выводы по проделанной работе.

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию.

Для всех динамических сцен необходимо задавать параметр скорости!

1) Создать класс Triangle и класс Point. Объявить список из n объектов класса Point, написать функцию, определяющую, какая из точек лежит внутри, а какая – снаружи треугольника.

2) Определить класс Rectangle и класс Point. Объявить список из n объектов класса Point. Написать функцию, определяющую, какая из точек лежит снаружи, а какая – внутри прямоугольника.

Код программы:

"""

Оконное приложение: класс Rectangle, класс Point и визуализация точек относительно прямоугольника.

Функции:

- ввод параметров прямоугольника и количества точек;
- генерация случайных точек;
- визуализация: точки внутри прямоугольника и снаружи;
- пауза/продолжение анимации;
- изменение скорости "на лету";
- сохранение скриншота Canvas в файл .ps в текущую директорию.

"""

```
from __future__ import annotations
```

```
import random
```

```
import time
```

```
from dataclasses import dataclass
```

```
from typing import List
```

```
import tkinter as tk
```

```
from tkinter import ttk, messagebox, filedialog
```

```
@dataclass
```

```
class Point:
```

```
    """Точка на плоскости."""
```

```
    x: float
```

```
    y: float
```

@dataclass

class Rectangle:

"""Прямоугольник, заданный левым верхним и правым нижним углом."""

x1: float

y1: float

x2: float

y2: float

def contains(self, point: Point) -> bool:

"""Проверка, лежит ли точка внутри прямоугольника (включая границу)."""

return self.x1 <= point.x <= self.x2 and self.y1 <= point.y <= self.y2

class RectPointsApp:

"""Главное окно приложения для работы с прямоугольником и точками."""

def __init__(self, root: tk.Tk) -> None:

self.root = root

self.root.title("Rectangle & Points")

self.canvas_width = 600

self.canvas_height = 400

self.canvas = tk.Canvas(

self.root,

width=self.canvas_width,

```
        height=self.canvas_height,

        bg="white",

    )

    self.canvas.grid(row=0, column=0, columnspan=4, padx=5, pady=5)

    # Параметры прямоугольника

    ttk.Label(self.root, text="x1").grid(row=1, column=0, sticky="w")

    self.entry_x1 = ttk.Entry(self.root, width=7)

    self.entry_x1.insert(0, "100")

    self.entry_x1.grid(row=1, column=1, sticky="w")

    ttk.Label(self.root, text="y1").grid(row=1, column=2, sticky="w")

    self.entry_y1 = ttk.Entry(self.root, width=7)

    self.entry_y1.insert(0, "100")

    self.entry_y1.grid(row=1, column=3, sticky="w")

    ttk.Label(self.root, text="x2").grid(row=2, column=0, sticky="w")

    self.entry_x2 = ttk.Entry(self.root, width=7)

    self.entry_x2.insert(0, "500")

    self.entry_x2.grid(row=2, column=1, sticky="w")

    ttk.Label(self.root, text="y2").grid(row=2, column=2, sticky="w")

    self.entry_y2 = ttk.Entry(self.root, width=7)

    self.entry_y2.insert(0, "300")

    self.entry_y2.grid(row=2, column=3, sticky="w")
```

Количество точек

```
ttk.Label(self.root, text="Количество точек").grid(  
    row=3,  
    column=0,  
    sticky="w",  
)
```

```
self.entry_n = ttk.Entry(self.root, width=7)
```

```
self.entry_n.insert(0, "50")
```

```
self.entry_n.grid(row=3, column=1, sticky="w")
```

Скорость (задержка в мс)

```
ttk.Label(self.root, text="Задержка (мс)").grid(row=3, column=2, sticky="w")
```

```
self.entry_delay = ttk.Entry(self.root, width=7)
```

```
self.entry_delay.insert(0, "50")
```

```
self.entry_delay.grid(row=3, column=3, sticky="w")
```

Кнопки управления

```
self.btn_draw_rect = ttk.Button(  
    self.root,
```

```
    text="Нарисовать прямоугольник",
```

```
    command=self.draw_rectangle,
```

```
)
```

```
self.btn_draw_rect.grid(row=4, column=0, columnspan=2, sticky="we", padx=2)
```

```
self.btn_start = ttk.Button(  
    self.root,
```

```
        text="Старт визуализации",
        command=self.start_visualization,
    )
    self.btn_start.grid(row=4, column=2, columnspan=2, sticky="we", padx=2)

    self.btn_pause = ttk.Button(
        self.root,
        text="Пауза / Продолжить",
        command=self.toggle_pause,
    )
    self.btn_pause.grid(row=5, column=0, columnspan=2, sticky="we", padx=2)

    self.btn_clear = ttk.Button(
        self.root,
        text="Очистить",
        command=self.clear_canvas,
    )
    self.btn_clear.grid(row=5, column=2, sticky="we", padx=2)

    self.btn_screenshot = ttk.Button(
        self.root,
        text="Скриншот",
        command=self.save_screenshot,
    )
    self.btn_screenshot.grid(row=5, column=3, sticky="we", padx=2)
```

```
self.rectangle: Rectangle | None = None
```

```
self.points: List[Point] = []
```

```
self.is_paused = False
```

```
self.is_running = False
```

```
def _read_rectangle(self) -> Rectangle | None:
```

```
    """Считать параметры прямоугольника из полей ввода."""
```

```
    try:
```

```
        x1 = float(self.entry_x1.get())
```

```
        y1 = float(self.entry_y1.get())
```

```
        x2 = float(self.entry_x2.get())
```

```
        y2 = float(self.entry_y2.get())
```

```
    except ValueError:
```

```
        messagebox.showerror("Ошибка", "Координаты прямоугольника должны быть  
числами.")
```

```
        return None
```

```
    if x2 <= x1 or y2 <= y1:
```

```
        messagebox.showerror("Ошибка", "x2 > x1 и y2 > y1 должны выполняться.")
```

```
        return None
```

```
    return Rectangle(x1, y1, x2, y2)
```

```
def draw_rectangle(self) -> None:
```

```
    """Нарисовать прямоугольник на Canvas."""
```

```
    rect = self._read_rectangle()
```

```
    if rect is None:
```


return

self.rectangle = rect

self.canvas.delete("all")

self.canvas.create_rectangle(

rect.x1,

rect.y1,

rect.x2,

rect.y2,

outline="black",

width=2,

)

def _generate_points(self, n: int) -> List[Point]:

"""Сгенерировать n случайных точек в пределах Canvas."""

return [

Point(

x=random.uniform(0, self.canvas_width),

y=random.uniform(0, self.canvas_height),

)

for _ in range(n)

]

def start_visualization(self) -> None:

"""Запустить визуализацию точек."""

if self.rectangle is None:

```

        self.draw_rectangle()

        if self.rectangle is None:

            return

    try:

        n_points = int(self.entry_n.get())

        delay_ms = int(self.entry_delay.get())

    except ValueError:

        messagebox.showerror("Ошибка", "Количество точек и задержка должны быть целыми.")

        return

    if n_points <= 0:

        messagebox.showerror("Ошибка", "Количество точек должно быть > 0.")

        return

    self.points = self._generate_points(n_points)

    self.is_paused = False

    self.is_running = True

    self.canvas.delete("all")

    rect = self.rectangle

    if rect is not None:

        self.canvas.create_rectangle(

            rect.x1,

            rect.y1,

            rect.x2,

```

```
rect.y2,  
outline="black",  
width=2,  
)
```

```
self._animate_points(delay_ms)
```

```
def _animate_points(self, delay_ms: int) -> None:
```

```
    """Пошаговая визуализация точек."""
```

```
    if not self.is_running:
```

```
        return
```

```
    if not self.points:
```

```
        self.is_running = False
```

```
        return
```

```
    if self.is_paused:
```

```
        self.root.after(100, lambda: self._animate_points(delay_ms))
```

```
        return
```

```
    point = self.points.pop(0)
```

```
    color = "green" if self.rectangle and self.rectangle.contains(point) else "red"
```

```
    radius = 3
```

```
    self.canvas.create_oval(  
        point.x - radius,
```

```
        point.y - radius,
```

```
    point.x + radius,  
    point.y + radius,  
    fill=color,  
    outline="",  
)
```

```
try:
```

```
    delay_ms = int(self.entry_delay.get())
```

```
except ValueError:
```

```
    delay_ms = 50
```

```
self.root.after(delay_ms, lambda: self._animate_points(delay_ms))
```

```
def toggle_pause(self) -> None:
```

```
    """Поставить визуализацию на паузу или продолжить."""
```

```
    if not self.is_running:
```

```
        return
```

```
    self.is_paused = not self.is_paused
```

```
def clear_canvas(self) -> None:
```

```
    """Очистить Canvas."""
```

```
    self.canvas.delete("all")
```

```
    self.is_running = False
```

```
    self.is_paused = False
```

```
def save_screenshot(self) -> None:
```

```
"""Сохранить содержимое Canvas в PostScript-файл."""
```

```
filename = filedialog.asksaveasfilename(
```

```
    defaultextension=".ps",
```

```
    filetypes=[("PostScript", "*.ps")],
```

```
    initialfile=f"rect_points_{int(time.time())}.ps",
```

```
)
```

```
if not filename:
```

```
    return
```

```
self.canvas.postscript(file=filename)
```

```
messagebox.showinfo("Сохранено", f"Скриншот сохранён в файл: \n{filename}")
```

```
def main() -> None:
```

```
    """Точка входа."""
```

```
    root = tk.Tk()
```

```
    app = RectPointsApp(root)
```

```
    root.mainloop()
```

```
if __name__ == "__main__":
```

```
    main()
```